36. Design Azure Key Vault

Key Terms:

Term	Full Name	What it does
HSM	Hardware Security Module	Tamper-resistant device protecting keys
FIPS 140-2	Federal Information Processing Standard	Security certification for crypto modules
KEK	Key Encryption Key	Key that encrypts other keys
DEK	Data Encryption Key	Key that encrypts actual data
RBAC	Role-Based Access Control	Permissions by role assignment
Managed Identity	Azure AD identity	Automatic credential management for Azure services
Soft Delete	Recoverable deletion	Deleted secrets recoverable for retention period

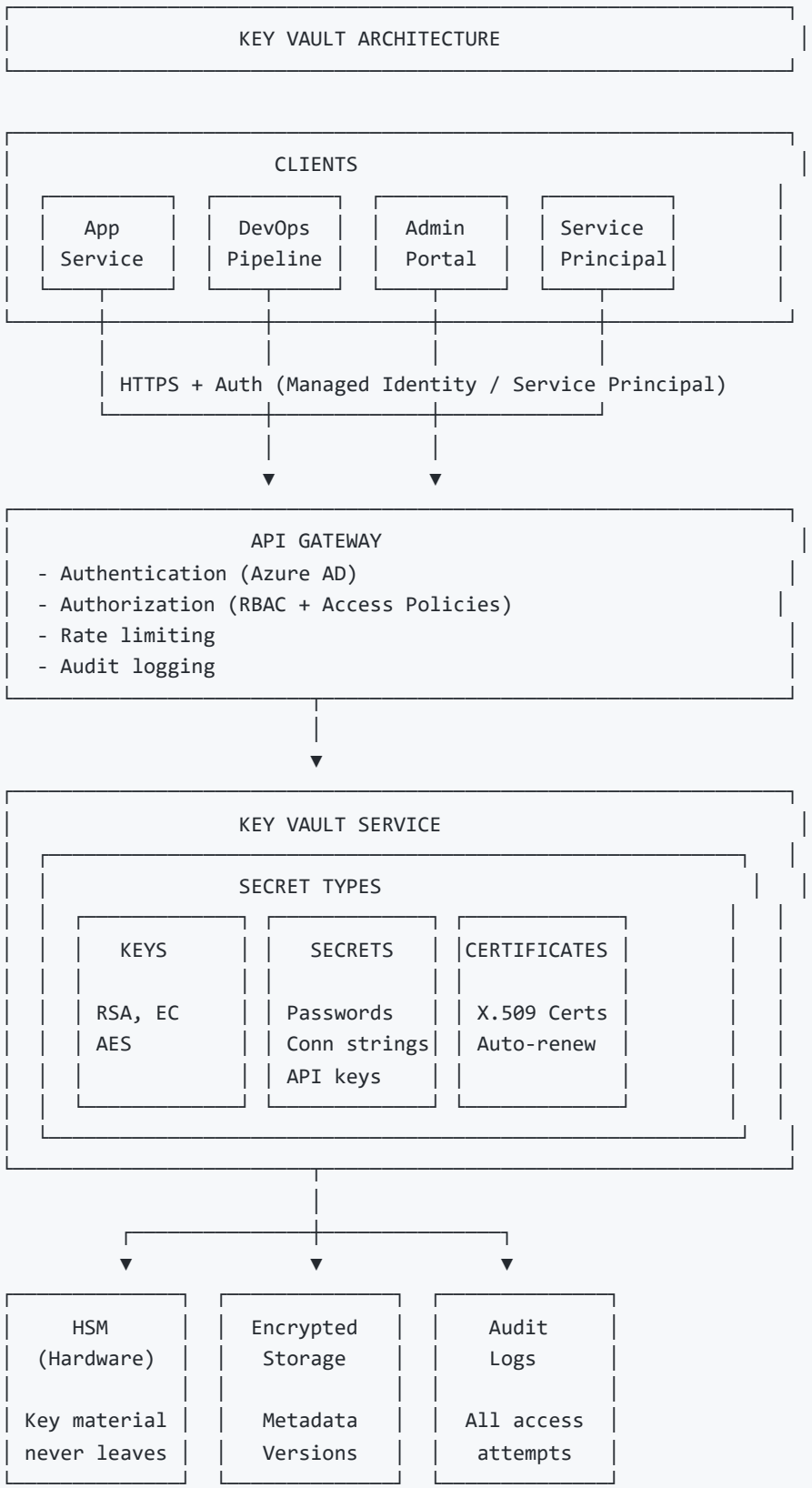
Problem Statement

Design a secure secrets management service that stores and retrieves encryption keys, certificates, and secrets with fine-grained access control.

Clarifying Questions to Ask

1. **Secret types**: Keys, certificates, connection strings, API keys?
2. **Scale**: How many secrets? Access frequency?
3. **Compliance**: FIPS 140-2, HSM requirements?
4. **Access control**: Who can access what?
5. **Rotation**: Automatic key rotation?
6. **Audit**: Track all access attempts?
7. **Availability**: Multi-region replication?

High-Level Design



API Design

Secret APIs

Method	Endpoint	Description
`PUT`	`/api/v1/secrets/{secretName}`	Create or update a secret
`GET`	`/api/v1/secrets/{secretName}`	Get secret value
`GET`	`/api/v1/secrets/{secretName}/versions`	List secret versions
`GET`	`/api/v1/secrets/{secretName}/{version}`	Get specific version
`DELETE`	`/api/v1/secrets/{secretName}`	Soft-delete a secret
`POST`	`/api/v1/secrets/{secretName}/recover`	Recover deleted secret

Key APIs (Cryptographic Operations)

Method	Endpoint	Description
`POST`	`/api/v1/keys/{keyName}/create`	Create a cryptographic key
`POST`	`/api/v1/keys/{keyName}/encrypt`	Encrypt data
`POST`	`/api/v1/keys/{keyName}/decrypt`	Decrypt data
`POST`	`/api/v1/keys/{keyName}/sign`	Sign data
`POST`	`/api/v1/keys/{keyName}/verify`	Verify signature
`POST`	`/api/v1/keys/{keyName}/rotate`	Rotate key

Certificate APIs

Method	Endpoint	Description
`POST`	`/api/v1/certificates/{certName}/create`	Create/import certificate
`GET`	`/api/v1/certificates/{certName}`	Get certificate
`POST`	`/api/v1/certificates/{certName}/renew`	Renew certificate

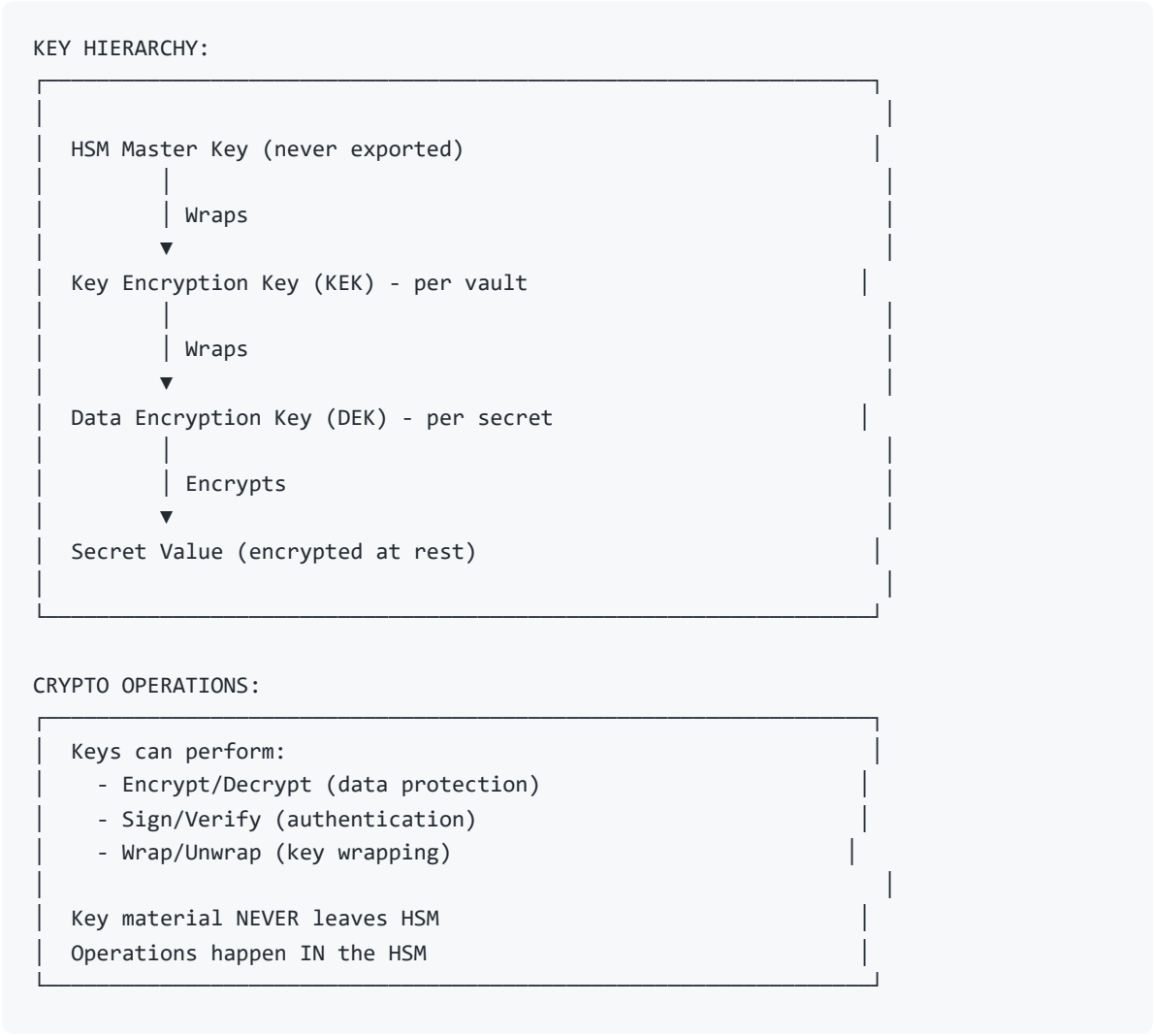
Access Policy APIs

Method	Endpoint	Description
`PUT`	`/api/v1/policies/{principalId}`	Set access policy
`GET`	`/api/v1/audit/logs?objectName={name}`	Get audit trail

Key Components

Component	Purpose	Technology
Auth Service	Verify caller identity	Azure AD
Policy Engine	Check permissions	RBAC + Access Policies
HSM	Protect key material	FIPS 140-2 Level 2/3
Secret Store	Encrypted secret storage	Encrypted database
Version Store	Secret history	Append-only storage
Audit Log	Track all operations	Immutable log

Encryption Architecture



Access Control

```
RBAC + ACCESS POLICIES:
┌──────────────────────────────────────────────────────────────┐
│  Management Plane (RBAC):                                    │
│    - Who can create/delete vaults                            │
│    - Who can set access policies                             │
│    - Roles: Owner, Contributor, Reader                       │
│                                                              │
│  Data Plane (Access Policies):                               │
│    - Who can read/write specific secrets                     │
│    - Granular permissions per principal                      │
│                                                              │
│  Example Policy:                                             │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Principal: app-service-prod                         │    │
│  │  Permissions:                                        │    │
│  │    Keys: Get, List                                   │    │
│  │    Secrets: Get (NOT Set, Delete)                    │    │
│  │    Certificates: Get                                 │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘

PRINCIPLE OF LEAST PRIVILEGE:
┌──────────────────────────────────────────────────────────────┐
│  - Apps: Only Get secrets they need                          │
│  - DevOps: Set secrets, no Delete                            │
│  - Admins: Full control, requires MFA                        │
│  - No one: Export private keys                               │
└──────────────────────────────────────────────────────────────┘
```

Secret Rotation

```
AUTOMATIC ROTATION:
┌──────────────────────────────────────────────────────────────┐
│  1. Key Vault triggers rotation (schedule or event)          │
│  2. Generate new secret value                                │
│  3. Update dependent service (e.g., SQL password)            │
│  4. Store new version in Key Vault                           │
│  5. Apps automatically get new value (no redeploy)           │
│                                                              │
│  Version History:                                            │
│    secret/db-password                                        │
│      ├── v1 (expired)                                        │
│      ├── v2 (expired)                                        │
│      └── v3 (current) ← Apps get this                        │
└──────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Caching**: Apps should cache secrets (with TTL)

- **Rate limits**: Throttle excessive requests
- **Geo-replication**: Replicate to DR region
- **Soft delete**: Recover accidentally deleted secrets
- **Purge protection**: Prevent permanent deletion

Trade-offs

Decision	Option A	Option B	Recommendation
HSM	Software	Hardware	**Hardware** - compliance
Auth	API keys	Managed Identity	**Managed Identity** - no secrets
Caching	None	Client-side	**Client-side** - reduce latency
Replication	Single region	Multi-region	**Multi** - availability

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
HSM	Hardware security for key material protection	Tech Guide

How to Explain It Well

> **Opening**: "Key Vault provides secure secrets storage with HSM protection. Key material never leaves the HSM - crypto operations happen inside."

> **Key talking points**:
>
> - "Key hierarchy: HSM Master → KEK → DEK → encrypted secret"
> - "Managed Identity for auth - apps don't need secrets to get secrets"
> - "Access policies: least privilege, apps only get what they need"
> - "Automatic rotation with version history"
> - "Audit log captures every access attempt"
